# Housemate Model Service

## Results Document

| | |
|---|---|
| **Date:** | 29 / Sep / 2026 |
| **Author:** | Mauricio Gidi |
| **Reviewer(s):** | Zachary Tunison |

## Introduction

This document summarizes the results of designing and implementing the Housemate Model Service. It evaluates how the design supported implementation, identifies opportunities for improvement, documents design-to-implementation differences, and summarizes the Assignment 2 peer design review.

## 1. Design and Implementation Results

### 1.1 Effect of the Design on Implementation

Creating the design made implementation easier because the class diagram established the main structure before coding. The Service and HousemateModelService boundary, the House–Room–Device structure, the Occupant and Device generalizations, and the KnowledgeGraph dependency translated directly into the implementation. This reduced structural decisions during coding. The pre-implementation class dictionary and testing details were less useful because they were not specified at the same level of detail as the class diagram.

### 1.2 Opportunities to Improve the Design

The design could have been more precise before implementation about method signatures, exception behavior, and testing. The pre-implementation class dictionary did not fully specify the parameters, return types, or behavior of all methods. The design also could have defined earlier how the CLI handles malformed commands and service errors, and how device state is represented in the KnowledgeGraph. Specifying these details before implementation would have reduced implementation-level decisions and provided a clearer testing strategy.

## 2. Changes Made During Implementation

### 2.1 Design-to-Implementation Changes

The implementation did not require substantive deviations from the revised design. Concrete HashMap collections, private entity lookup and type-validation helpers, and KnowledgeGraph predicate naming were resolved during implementation, but these decisions did not alter the classes, public Service interface, associations, or core behaviors defined by the design.

## 3. Design Review Results

### 3.1 Impact of the Design Review

The design review improved the design by drawing attention to exception specificity, testing, use-case scope, and the boundary between the model service and test infrastructure. The most direct refinement was the use of more specific exception types instead of relying only on broad service exceptions. The review also confirmed the decision to keep the TestDriver outside the main model class diagram and highlighted that the testing strategy could have been documented more completely.

### 3.2 Feedback Received

#### 3.2.1 Feedback from Zachary Tunison

| Feedback received |
| --- |
| Reconsider broad exception handling and use more specific exception types where appropriate.<br>Consider whether the testing class should appear in the class diagram.<br>Keep the use-case diagram focused on the scope of the Housemate Model Service rather than the broader system architecture.<br>Include a Testing section in the design document. |

| Response / design impact |
| --- |
| The exception design was refined to use EntityNotFoundException, DuplicateEntityException, and InvalidStateException for model-service errors, with InvalidCommandException for CLI errors. The TestDriver remains outside the main model class diagram because it exercises the component rather than forming part of the model itself. The use-case scope was reviewed against the Model Service requirements and remains focused on configuration, device-state, and energy-use operations. A dedicated Testing section is present in the revised design, although its strategy could have been specified more completely. |

## 4. Feedback Provided to Peer Review Partners

### 4.1 Feedback Provided to Zachary Tunison

- Use the KnowledgeGraph as the clear source of truth for sensor state if state might otherwise be duplicated between SensorStatus and the KnowledgeGraph.
- Have HouseModel implement the Service interface so that CommandAPI depends on Service rather than the concrete HouseModel implementation.
- Reconsider composition between HouseModel and House or Occupant. Regular associations may be more appropriate unless HouseModel explicitly owns their lifecycle.
- Keep the test class outside the main class diagram.
- Rearrange the House–Occupant and Room–Sensor associations to make the diagram easier to read.